Apply Hash Functions and Integrity

Password Hashing System · HMAC Message Authentication · File Integrity Verification

Python implementations verified against RFC test vectors and the Python standard library — 19 of 19 tests passed

Project Name	Healthcare IAM Operations Simulation
Organization	Horizon Valley Medical Center (HVMC)
Rubric	Cryptography 1 — Apply Hash Functions and Integrity (Semester 4, Week 10)
Code	`cryptography_1/hash_integrity/` : `password_hashing.py` , `hmac_auth.py` , `file_integrity.py` , `test_and_demo.py` , `sample_export/` , `results.txt`
Environment	Python 3.9.6 on Darwin arm64, 2026-10-14 14:06; argon2/bcrypt: installed
Prepared By	Sabitha S Karanavar
Report Date	October 14, 2026

Rubric Cross-Reference

Rubric requirement	Section	Evidence
Development and documentation of hash function implementations	2–4, App. A	PBKDF2 and HMAC written from scratch and checked against RFC 7914 / RFC 4231 vectors; 19 passing tests; code excerpts
Secure password hashing system using modern algorithms	2	Argon2id (RFC 9106) and PBKDF2-HMAC-SHA256 (NIST SP 800-132) behind one interface; per-user salt, pepper, PHC-format strings, constant-time verify, automatic parameter upgrade, NIST SP 800-63B policy
Implementation of HMAC for message authentication	3	HMAC-SHA256 from scratch; signed LIS→EHR message protocol with key ids, timestamps and nonces
File integrity verification system	4	Command-line FIM tool: streamed SHA-256, HMAC-sealed manifest, Merkle tree with inclusion proofs
Documentation of implementation security properties	2.5, 3.4, 4.4, 5	Property tables per component, algorithm status, best practices, limitations

1. Overview

The three components share one foundation, the SHA-256 hash function (implemented from scratch in the Week 9 report). Each adds what a bare hash lacks for a particular job:

Component	What it adds to SHA-256	HVMC use	HIPAA Security Rule
Password hashing	Random salt, tunable work factor, memory hardness (Argon2id), secret pepper	Patient-portal accounts	§164.312(d) authentication; §164.308(a)(5)(ii)(D) password management
HMAC	A secret key, so only key holders can produce a valid tag	LIS → EHR lab messages	§164.312(c)(2) authenticate ePHI; §164.312(e)(2)(i) integrity controls

Component	What it adds to SHA-256	HVMC use	HIPAA Security Rule
File integrity	A sealed baseline of every file and a Merkle root	EHR exports and audit logs	§164.312(c)(1) integrity; §164.312(b) audit controls

HVMC staff sign in with Microsoft Entra ID, so staff passwords are handled by Microsoft. The password system documented here is for accounts HVMC stores itself, such as patient-portal users.

2. Secure Password Hashing System

2.1 Design

Design element	Implementation
Algorithms	**Argon2id** (RFC 9106; memory 19 MiB, 2 passes, 1 lane, the OWASP baseline) via `argon2-cffi`; **PBKDF2-HMAC-SHA256** with 600,000 iterations as the dependency-free fallback. `make_hasher()` picks Argon2id when it is installed.
Salt	16 random bytes per password from the OS CSPRNG (`secrets`), stored with the hash
Pepper (optional)	A 32-byte server secret applied with HMAC-SHA256 before hashing. Kept in Azure Key Vault, never in the database.
Storage format	Self-describing PHC-style strings, so the algorithm and parameters travel with each hash: `$pbkdf2-sha256$i=600000$<salt>$<hash>` and `$argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash>`
Verification	Recompute with the stored salt and parameters; compare with `hmac.compare_digest` (constant time)
Upgrades	`needs_rehash()` flags hashes made with older parameters; they are replaced on the user's next successful login
Password policy	NIST SP 800-63B: minimum 15 characters (password-only accounts), maximum 64, screened against a list of common passwords and HVMC/username terms, no forced composition rules or periodic resets

2.2 Algorithms and mathematics

PBKDF2 (RFC 8018) repeatedly applies HMAC-SHA256 (PRF) and XORs the results. For output block i:

$$U_1 = \mathrm{PRF}(P, S \,\|\, \mathrm{INT}(i)), \quad U_j = \mathrm{PRF}(P, U_{j-1}), \quad T_i = U_1 \oplus U_2 \oplus \ldots \oplus U_c$$

The iteration count c sets the work per password check. The implementation computes this from scratch and matches both RFC 7914 test vectors and Python's `hashlib.pbkdf2_hmac`.

Argon2id (winner of the 2015 Password Hashing Competition) fills a block of memory (here 19 MiB) with values derived from the password and salt, then reads it back in both data-independent and data-dependent order. Requiring a large amount of memory per computation, not just time, is called **memory hardness**. The "id" variant combines the side-channel resistance of Argon2i with the strength of Argon2d.

```
def pbkdf2_sha256(password, salt, iterations, dklen=32):
    """RFC 8018 section 5.2.  T_i = U_1 xor U_2 xor ... xor U_c
    with U_1 = PRF(P, S || INT(i)) and U_j = PRF(P, U_{j-1}); PRF = HMAC-SHA256."""
    prf = HMAC(password)
    blocks = []
    for i in range(1, -(-dklen // 32) + 1):
        u = prf.digest(salt + struct.pack(">I", i))
        t = int.from_bytes(u, "big")
        for _ in range(iterations - 1):
            u = prf.digest(u)
            t ^= int.from_bytes(u, "big")
        blocks.append(t.to_bytes(32, "big"))
    return b"".join(blocks)[:dklen]
```

```
def _prepare(self, password):
        pw = password.encode("utf-8")
```

```
        if self.pepper:
            pw = std_hmac.new(self.pepper, pw, hashlib.sha256).digest()
        return pw

def hash(self, password):
        salt = secrets.token_bytes(16)
        dk = hashlib.pbkdf2_hmac("sha256", self._prepare(password), salt, self.iterations)
        return f"$pbkdf2-sha256$i={self.iterations}${_b64(salt)}${_b64(dk)}"

def verify(self, stored, password):
        try:
            _, alg, cost, salt, dk = stored.split("$")
            iterations = int(cost.split("=")[1])
        except (ValueError, IndexError):
            return False
        if alg != "pbkdf2-sha256":
            return False
        candidate = hashlib.pbkdf2_hmac("sha256", self._prepare(password), _unb64(salt), iterations)
        return std_hmac.compare_digest(candidate, _unb64(dk))

def needs_rehash(self, stored):
        """True if the stored hash used fewer iterations than current policy:
        re-hash it the next time the user logs in successfully."""
        try:
            return int(stored.split("$")[2].split("=")[1]) < self.iterations
        except (ValueError, IndexError):
            return True
```

```
def hash(self, password):
        return self._ph.hash(self._prepare(password))

def verify(self, stored, password):
        try:
            return self._ph.verify(stored, self._prepare(password))
        except argon2.exceptions.VerificationError:
            return False
        except argon2.exceptions.InvalidHashError:
            return False
```

2.3 Working results

PBKDF2 stored value 1	`$pbkdf2-sha256$i=600000$U138JP0tRjEqUVeO3qFDpQ$tTldKOx/rYcmorAvvI2OJVwx8bLgO6EXaK9JsYvScD4`
PBKDF2 stored value 2 (same password)	`$pbkdf2-sha256$i=600000$buH8CrS0lNbCC2j76NHpTQ$pQB+IVRNRXTgnvX7K83+qwc0vExB8Gg2VUTnrQumwBk`
Argon2id stored value	`$argon2id$v=19$m=19456,t=2,p=1$Kgs8P7X+92D8hhEG0JVKCQ$8MBtzCWoQ6D0Ky9LLwszgLtKPDiYOf7VK2p7oXkigdg` (15 ms)
Unique per storage	**True — same password gives different stored values because of the salt**
Pepper	Verifies with pepper: **True**; verifies from database copy alone: **False**
Automatic upgrade	Legacy `i=100000` flagged by `needs_rehash()` and replaced with `i=600000` at next login

Measured time per hash on this Mac (one core), used to set parameters so a login check stays well under the commonly used ~1 second budget:

Algorithm and parameters	Time per hash
MD5 (unsuitable)	0.0002 ms
SHA-256 (unsuitable)	0.0002 ms

Algorithm and parameters	Time per hash
PBKDF2-HMAC-SHA256, 600,000 iterations	139.4820 ms
bcrypt, cost 12	166.9634 ms
Argon2id, 19 MiB, t=2, p=1	13.6873 ms

MD5 and SHA-256 are shown only as a reference point. They are general-purpose hashes designed for speed and are not used for passwords.

2.4 Password policy screening (NIST SP 800-63B)

Candidate password	Username	Result	Reason
`Password1`	—	**REJECT**	shorter than 15 characters; on the common/breached-password blocklist
`Winter2026!`	—	**REJECT**	shorter than 15 characters; on the common/breached-password blocklist
`Hvmc2026!`	—	**REJECT**	shorter than 15 characters; on the common/breached-password blocklist; contains HVMC context words
`Tr0ub4dor&3`	—	**REJECT**	shorter than 15 characters
`lisatran-portal-login`	lisa.tran	**REJECT**	contains the username
`aaaaaaaaaaaaaaaaaaa`	—	**REJECT**	repetitive characters
`copper kettle meadow sunrise`	—	**ACCEPT**	—
`my dog Biscuit hates thunderstorms`	—	**ACCEPT**	—

2.5 Security properties

Property	How the system provides it	Evidence
Passwords never stored	Only salt + derived hash are kept; the KDF is one-way	Stored values in 2.3
Uniqueness per account	Random 128-bit salt per hash	`test_unique_salts`
Tunable work factor	Iterations (PBKDF2) or memory/passes (Argon2id) recorded in each string	Measured costs, 2.3
Memory hardness	Argon2id needs 19 MiB per computation	Argon2id tests
Separation of secrets	Pepper lives in Key Vault, so a database copy alone is not enough to verify	`test_pepper`
Timing-safe comparison	`hmac.compare_digest`	`verify()`
Crypto agility	Algorithm and parameters stored per hash; `needs_rehash` migrates users	`test_needs_rehash`

3. HMAC for Message Authentication

3.1 Construction

$$\mathrm{HMAC}(K, m) = H\big((K' \oplus \mathrm{opad}) \,\|\, H((K' \oplus \mathrm{ipad}) \,\|\, m)\big)$$

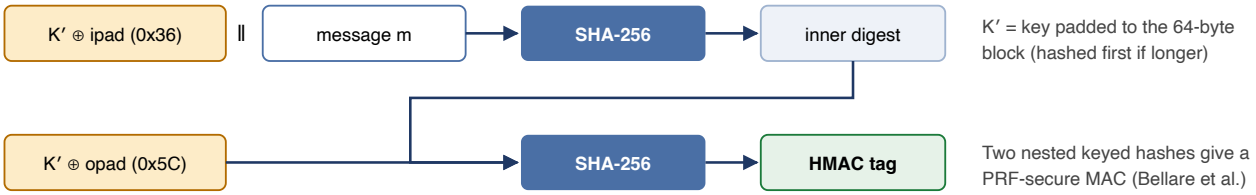


Figure 1 — HMAC-SHA256 as implemented in `hmac_auth.py`.

The nested structure is what makes HMAC safe with Merkle–Damgård hashes such as SHA-256. The Week 9 report showed that a bare `SHA-256(key ‖ message)` tag could be extended, while HMAC could not. The implementation precomputes the keyed inner and outer states once per key and copies them for each message, which is the same optimization production libraries use.

```
class HMAC:
    """From-scratch HMAC over any hashlib hash (SHA-256 by default).
    The keyed inner/outer states are computed once and copied per message,
    the same optimisation real libraries use."""

    def __init__(self, key, digestmod=hashlib.sha256):
        self.digestmod = digestmod
        block = digestmod().block_size              # 64 bytes for SHA-256
        if len(key) > block:
            key = digestmod(key).digest()           # long keys are hashed first
        key = key.ljust(block, b"\x00")
        self._inner = digestmod(bytes(b ^ IPAD for b in key))
        self._outer = digestmod(bytes(b ^ OPAD for b in key))

    def digest(self, message):
        inner = self._inner.copy()
        inner.update(message)
        outer = self._outer.copy()
        outer.update(inner.digest())
        return outer.digest()

def verify_tag(key, message, tag):
    """Constant-time comparison: the time taken does not depend on how many bytes match,
    so an attacker cannot guess a valid tag byte by byte."""
    return std_hmac.compare_digest(hmac_sha256(key, message), tag)
```

3.2 Authenticated message protocol (LIS → EHR)

Field	Purpose
`kid` (key id)	Selects the shared key; supports rotation and revocation without code changes
`ts` (timestamp)	Receiver accepts only messages within a 5-minute window
`nonce`	96-bit random value; receiver rejects any nonce seen before
`tag`	HMAC-SHA256 over canonical JSON (sorted keys, fixed separators) of header and body

```
def accept(self, msg, now=None):
        h = msg["header"]
        now = int(time.time()) if now is None else now
        if h["kid"] not in self.keys:
            raise AuthError("unknown or revoked key id")
        if not verify_tag(self.keys[h["kid"]], canonical(h, msg["body"]), bytes.fromhex(msg["tag"])):
            raise AuthError("invalid tag - message altered or wrong key")
        if abs(now - h["ts"]) > self.max_age:
            raise AuthError("message too old (outside 5-minute window)")
        if h["nonce"] in self.seen:
            raise AuthError("replayed message (nonce already used)")
        self.seen.add(h["nonce"])
        return msg["body"]
```

3.3 Working results

Tag for the genuine potassium-result message: `f72a7c4bcb9d49ac4facc2cbeba1b8421c353a3e383f015be283472eb0c84413`

Scenario	Receiver decision
1. genuine message	**ACCEPTED**
2. same message sent again (replay)	**REJECTED - replayed message (nonce already used)**
3. value changed 4.1 -> 7.1 in transit	**REJECTED - invalid tag - message altered or wrong key**
4. delivered 1 hour late	**REJECTED - message too old (outside 5-minute window)**
5. signed with a retired key id	**REJECTED - unknown or revoked key id**

The implementation matches all four RFC 4231 test cases used and Python's `hmac` module for SHA-256 and SHA-512.

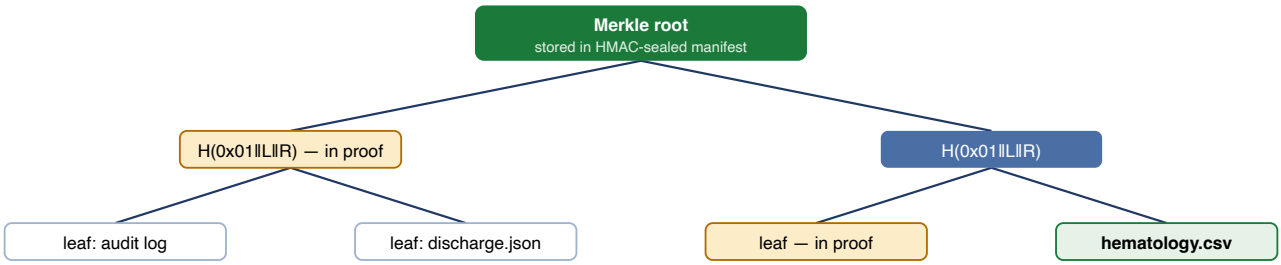
3.4 Security properties

Property	Provided by
Message integrity	Any change to header or body changes the tag
Data-origin authentication	Only holders of the shared key can produce a valid tag
Freshness / replay protection	Timestamp window plus nonce cache
Key lifecycle	Key ids allow rotation; removing a key id revokes it
Timing-safe verification	`hmac.compare_digest`
Not provided	Confidentiality (pair with AES-GCM or TLS) and non-repudiation (both sides share the key; use digital signatures for that)

4. File Integrity Verification System

4.1 Design

Step	What `file_integrity.py` does
Hash	SHA-256 of every file, streamed in 64 KiB chunks so file size does not matter (measured throughput: 3,068 MB/s)
Baseline	Manifest of path → digest and size, plus file count, timestamp, key id and Merkle root
Seal	HMAC-SHA256 over the whole manifest with a key held in Key Vault, so the manifest cannot be edited to hide a change
Verify	Re-hash and report MODIFIED, DELETED and ADDED files, plus whether the seal is intact
Prove	Merkle inclusion proof that one file belongs to a sealed baseline, without sharing every other file's hash



leaf = H(0x00 ‖ path ‖ SHA-256(file)). Proving one file needs only the orange sibling hashes (about $\log_2 n$), not the whole list.

Figure 2 — Merkle tree over the export folder. 0x00/0x01 prefixes separate leaf and node hashes.

```
def hash_file(path):
    """SHA-256 of a file without loading it all into memory."""
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(CHUNK), b""):
            h.update(chunk)
    return h.hexdigest()

def create_baseline(folder, key, key_id="fim-v1"):
    files = scan(folder)
    manifest = {"algorithm": "sha256", "created": time.strftime("%Y-%m-%dT%H:%M:%S"), "key_id": key_id,
                "file_count": len(files), "merkle_root": merkle_root(files), "files": files}
    manifest["hmac"] = hmac.new(key, _seal_bytes(manifest), hashlib.sha256).hexdigest()
    return manifest

def verify_baseline(folder, manifest, key):
    report = {"manifest_ok": hmac.compare_digest(
        hmac.new(key, _seal_bytes(manifest), hashlib.sha256).hexdigest(), manifest.get("hmac", ""))}
    old, new = manifest["files"], scan(folder)
    report["modified"] = sorted(p for p in old.keys() & new.keys() if old[p]["sha256"] != new[p]["sha256"])
    report["deleted"] = sorted(old.keys() - new.keys())
    report["added"] = sorted(new.keys() - old.keys())
    report["merkle_root_now"] = merkle_root(new)
    report["clean"] = report["manifest_ok"] and not (report["modified"] or report["deleted"] or report["added"])
    return report
```

```
export HVMC_FIM_KEY=<64 hex characters from Key Vault>
python3 file_integrity.py baseline sample_export manifest.json
python3 file_integrity.py verify   sample_export manifest.json     # exit code 0 = clean, 2 = alert
```

4.2 Working results on the sample HVMC export (5 files)

Change made	Result	Seal	Modified	Deleted	Added
A. no changes	**Clean**	OK	—	—	—
B. timestamps changed only (file copied / touched)	**Clean**	OK	—	—	—
C. one lab value edited (4.1 -> 7.1)	**ALERT**	OK	lab_results/2026-10-12_chemistry.csv	—	—
D. audit log deleted and an unexpected file added	**ALERT**	OK	lab_results/2026-10-12_chemistry.csv	audit/ehr_access_2026-10-12.log	ehr_exports/unexpected_file.json
E. manifest edited to match the changed lab file (no HMAC key)	**ALERT**	**FAILED**	—	audit/ehr_access_2026-10-12.log	ehr_exports/unexpected_file.json

Merkle proof for `lab_results/2026-10-12_hematology.csv`: 3 sibling hashes; genuine digest verifies: **True**; altered digest verifies: **False**.

4.3 Operating it at HVMC

- Run `verify` on a schedule (for example hourly for audit logs, daily for exports) and send any non-zero exit code to the Security Monitoring Portal (Anthony Ruiz, Security Officer).
- Store the manifest separately from the monitored data, ideally in immutable (WORM) Azure Blob storage, and the HMAC key in Key Vault.
- Re-baseline only after an approved change, so the baseline always reflects an authorized state.
- Microsoft Defender for Servers also offers built-in file integrity monitoring; this tool shows the underlying technique.

4.4 Security properties

Property	Provided by
Content-based detection	SHA-256 of file contents; metadata-only changes do not raise false alerts (row B)
Detects edits, deletions, additions	Set comparison of old and new manifests (rows C–D)
Tamper-evident baseline	HMAC seal over the manifest (row E)
Efficient proofs	Merkle tree with domain-separated leaf and node hashes
Not provided	Prevention or recovery (pair with backups and immutable storage); detection between scheduled runs

5. Security Properties Summary and Best Practices

5.1 Hash algorithm status

Algorithm	Status	Notes
MD5	**Broken**	Practical collisions since 2004; not for any security use
SHA-1	**Deprecated**	Public collision 2017 ("SHAttered"); NIST plans to retire it by 2030
SHA-256 / SHA-512	**Approved**	FIPS 180-4; used here for files, HMAC and PBKDF2
SHA-3, BLAKE2	**Approved / strong**	Not Merkle–Damgård, so no length-extension issue
Argon2id	**Recommended for passwords**	RFC 9106; OWASP first choice
PBKDF2-HMAC-SHA256	**Approved for passwords**	NIST SP 800-132; needed where FIPS-validated modules are required

5.2 Best-practice checklist

#	Practice	Where applied
1	Use a dedicated password-hashing algorithm (Argon2id, or PBKDF2 at ≥ 600,000 iterations), never a general-purpose hash	`make_hasher()`
2	Unique random salt per password; store parameters with the hash	PHC-format strings
3	Keep a pepper and HMAC/FIM keys in Key Vault, separate from data	Design, 2.1 and 4.1
4	Compare secrets in constant time	`compare_digest` everywhere
5	Use HMAC (not hash(key ‖ message)) for message authentication; include freshness	Section 3
6	Seal integrity baselines and store them apart from monitored files	Section 4
7	Review parameters yearly and migrate with `needs_rehash`	Section 2.3
8	Follow NIST SP 800-63B for password rules; add MFA	Section 2.4; Entra security defaults
9	Use vetted libraries in production; keep from-scratch code for learning and testing	From-scratch code verified against stdlib

5.3 Limitations

- The from-scratch PBKDF2 and HMAC are for demonstration and verification; production paths call `hashlib`, `hmac` and `argon2-cffi`.
- The nonce cache in the message receiver is in memory; a production service would use a shared store with expiry.
- Python cannot reliably erase keys from memory; Key Vault or an HSM avoids exposing keys to application memory.
- Argon2id requires the optional `argon2-cffi` package; on a plain Python 3.9 install the system falls back to PBKDF2 automatically.

6. Conclusion

The project delivers three working, tested hash-based controls for HVMC. The password system stores only salted, parameter-tagged Argon2id or PBKDF2 hashes, supports a Key Vault pepper and automatic upgrades, and screens passwords to NIST SP 800-63B. HMAC-SHA256, implemented from scratch and matched to RFC 4231, authenticates LIS-to-EHR messages and rejects altered, replayed, late or wrongly keyed messages. The file integrity system detects edited, deleted and added files in an EHR export, keeps its baseline tamper-evident with an HMAC seal, and supports Merkle proofs. All 19 tests pass.

References

- RFC 9106, *Argon2 Memory-Hard Function for Password Hashing* (2021).
- RFC 8018, *PKCS #5: Password-Based Cryptography Specification v2.1*; RFC 7914 (PBKDF2-HMAC-SHA256 test vectors).
- RFC 2104, *HMAC: Keyed-Hashing for Message Authentication*; RFC 4231 (HMAC-SHA256 test vectors).
- NIST FIPS 180-4, *Secure Hash Standard*; NIST FIPS 198-1, *The Keyed-Hash Message Authentication Code*.
- NIST SP 800-132, *Recommendation for Password-Based Key Derivation*; NIST SP 800-63B, *Digital Identity Guidelines: Authentication*.
- OWASP Password Storage Cheat Sheet.
- R. Merkle, "A Digital Signature Based on a Conventional Encryption Function," CRYPTO 1987.
- 45 CFR §164.312(b), (c), (d), (e).

Appendix A — How to Run and Test Results

```
cd "cryptography_1/hash_integrity"
python3 test_and_demo.py                # all tests + 6 demos; writes results.txt, results.json
python3 -m pip install argon2-cffi bcrypt && python3 test_and_demo.py   # optional: enables Argon2id tests
```

Area	Test	What it checks	Result
HMAC	`test_matches_stdlib`	Equal to Python's hmac module (SHA-256 and SHA-512, keys 0–200 bytes)	**PASS**
HMAC	`test_rfc4231_vectors`	RFC 4231 HMAC-SHA256 test cases 1, 2, 3 and 6	**PASS**
HMAC	`test_verify_tag`	Constant-time check accepts genuine tags; rejects changed message or key	**PASS**
PBKDF2	`test_matches_hashlib`	From-scratch PBKDF2 equals hashlib.pbkdf2_hmac	**PASS**
PBKDF2	`test_rfc7914_vectors`	RFC 7914 PBKDF2-HMAC-SHA256 vectors (1 and 80,000 iterations)	**PASS**
Password store	`test_malformed`	Corrupted stored value rejected safely	**PASS**
Password store	`test_needs_rehash`	Hashes below current cost policy are flagged for upgrade	**PASS**
Password store	`test_pepper`	Peppered hash only verifies with the server-side pepper	**PASS**
Password store	`test_round_trip_and_wrong_password`	Correct password verifies; one-character change fails	**PASS**
Password store	`test_unique_salts`	Same password stored twice gives different values	**PASS**
Argon2id	`test_needs_rehash_on_parameter_change`	Argon2id hash with older parameters flagged for upgrade	**PASS**
Argon2id	`test_round_trip_format_and_pepper`	Argon2id PHC format, verify, unique salts, pepper	**PASS**
Policy	`test_accepts_passphrase`	Long passphrase accepted; no composition rules	**PASS**
Policy	`test_rejects_weak`	Policy rejects short, listed, context-word, username, repetitive	**PASS**
HMAC protocol	`test_rejections`	Replay, tampering, expired timestamp, unknown key id rejected	**PASS**
HMAC protocol	`test_valid`	Genuine signed LIS message accepted	**PASS**
File integrity	`test_clean_and_changes`	No alert on metadata-only change; detects modified, deleted, added	**PASS**
File integrity	`test_manifest_tamper_detected`	Edited manifest or wrong key detected	**PASS**
File integrity	`test_merkle_proof`	Every file's Merkle proof verifies; altered digest fails	**PASS**

A.1 Program output (results.txt)

```
HVMC Hash Functions and Integrity — test and demo run
Python 3.9.6 on Darwin arm64, 2026-10-14 14:06; argon2/bcrypt: installed
```

```
=== Unit tests ===
test_needs_rehash_on_parameter_change (__main__.TestArgon2id) ... ok
test_round_trip_format_and_pepper (__main__.TestArgon2id) ... ok
test_rejections (__main__.TestAuthenticatedMessages) ... ok
test_valid (__main__.TestAuthenticatedMessages) ... ok
test_clean_and_changes (__main__.TestFileIntegrity) ... ok
test_manifest_tamper_detected (__main__.TestFileIntegrity) ... ok
test_merkle_proof (__main__.TestFileIntegrity) ... ok
test_matches_stdlib (__main__.TestHMAC) ... ok
test_rfc4231_vectors (__main__.TestHMAC) ... ok
test_verify_tag (__main__.TestHMAC) ... ok
test_matches_hashlib (__main__.TestPBKDF2) ... ok
test_rfc7914_vectors (__main__.TestPBKDF2) ... ok
test_malformed (__main__.TestPasswordHasher) ... ok
test_needs_rehash (__main__.TestPasswordHasher) ... ok
test_pepper (__main__.TestPasswordHasher) ... ok
test_round_trip_and_wrong_password (__main__.TestPasswordHasher) ... ok
test_unique_salts (__main__.TestPasswordHasher) ... ok
test_accepts_passphrase (__main__.TestPolicy) ... ok
test_rejects_weak (__main__.TestPolicy) ... ok

----------------------------------------------------------------------
Ran 19 tests in 0.194s

OK
tests run: 19, failures: 0, errors: 0

=== Demo 1: secure password storage format ===
stored hash 1: $pbkdf2-sha256$i=600000$U138JP0tRjEqUVeO3qFDpQ$tTldKOx/rYcmorAvvI2OJVwx8bLgO6EXaK9JsYvScD4
stored hash 2: $pbkdf2-sha256$i=600000$buH8CrS0lNbCC2j76NHpTQ$pQB+IVRNRXTgnvX7K83+qwc0vExB8Gg2VUTnrQumwBk
same password, different salts -> different stored values: True
verify correct password: True; verify wrong password: False
cost of one login check at 600,000 iterations: 140 ms
Argon2id stored hash: $argon2id$v=19$m=19456,t=2,p=1$Kgs8P7X+92D8hhEG0JVKCQ$8MBtzCWoQ6D0Ky9LLwszgLtKPDiYOf7VK2p7oXkigdg
Argon2id verify correct: True; wrong: False; 15 ms

=== Demo 2: cost per password-hash computation (this Mac, one core) ===
MD5 (unsuitable)                              0.0002 ms per hash
SHA-256 (unsuitable)                          0.0002 ms per hash
PBKDF2-HMAC-SHA256, 600,000 iterations      139.4820 ms per hash
bcrypt, cost 12                             166.9634 ms per hash
Argon2id, 19 MiB, t=2, p=1                   13.6873 ms per hash
A purpose-built password hash is deliberately slow (and Argon2id also memory-hard), which is what
makes large-scale guessing against a stolen database expensive. Fast hashes are for files, not passwords.

=== Demo 3: pepper and automatic cost upgrade ===
with pepper (kept in Key Vault): verify ok = True
database copy alone, no pepper: verify ok = False
legacy hash: i=100000 -> needs_rehash: True
after next successful login: i=600000 -> needs_rehash: False

=== Demo 4: NIST SP 800-63B password policy screening ===
'Password1'                          REJECT: shorter than 15 characters; on the common/breached-password blocklist
'Winter2026!'                        REJECT: shorter than 15 characters; on the common/breached-password blocklist
'Hvmc2026!'                          REJECT: shorter than 15 characters; on the common/breached-password blocklist;
contains HVMC context words
'Tr0ub4dor&3'                        REJECT: shorter than 15 characters
'lisatran-portal-login'              REJECT: contains the username
'aaaaaaaaaaaaaaaaaaaa'               REJECT: repetitive characters
'copper kettle meadow sunrise'       ACCEPT
'my dog Biscuit hates thunderstorms' ACCEPT

=== Demo 5: HMAC-authenticated LIS -> EHR messages ===
tag: f72a7c4bcb9d49ac4facc2cbeba1b8421c353a3e383f015be283472eb0c84413
1. genuine message                     ACCEPTED
2. same message sent again (replay)    REJECTED - replayed message (nonce already used)
3. value changed 4.1 -> 7.1 in transit REJECTED - invalid tag - message altered or wrong key
4. delivered 1 hour late               REJECTED - message too old (outside 5-minute window)
5. signed with a retired key id        REJECTED - unknown or revoked key id

=== Demo 6: file integrity monitoring of an EHR export folder ===
baseline: 5 files in 0.5 ms; merkle root 413273348085ece2add4d23a…; sealed with HMAC
A. no changes
    -> clean=True manifest_ok=True modified=[] deleted=[] added=[]
B. timestamps changed only (file copied / touched)
```

```
    -> clean=True manifest_ok=True modified=[] deleted=[] added=[]
C. one lab value edited (4.1 -> 7.1)
    -> clean=False manifest_ok=True modified=['lab_results/2026-10-12_chemistry.csv'] deleted=[] added=[]
D. audit log deleted and an unexpected file added
    -> clean=False manifest_ok=True modified=['lab_results/2026-10-12_chemistry.csv'] deleted=['audit/ehr_access_2026-10-
12.log'] added=['ehr_exports/unexpected_file.json']
E. manifest edited to match the changed lab file (no HMAC key)
    -> clean=False manifest_ok=False modified=[] deleted=['audit/ehr_access_2026-10-12.log'] added=
['ehr_exports/unexpected_file.json']
F. Merkle proof for lab_results/2026-10-12_hematology.csv: 3 sibling hashes; genuine file verifies: True; altered digest
verifies: False
G. SHA-256 throughput: 3,068 MB/s (100 MB hashed in 33 ms)
```

Prepared for: Sabitha — Healthcare IAM Operations Home Lab | Cryptography 1 rubric — Apply Hash Functions and Integrity (Semester 4, Week 10)